

在浏览器沙箱里，跑完整条媒体管线

Container 解析 → Demux → WebCodecs 解码 → Canvas / AudioWorklet 渲染，全部在浏览器里完成。零构建、纯 ESM、零第三方运行时依赖——不往页面塞一个几十 MB 的 wasm 运行时，只把你真正需要的容器模块 import 进来。

打开在线演示

npm i pure-webcodecs-player



970 / 970 单元测试通过

16 容器与协议模块

0 第三方运行时依赖

15 步 真机 e2e 自断言

WHY — 换一条更浏览器的路

别人让你先下几十 MB 运行时，我们只让你 import 一行

解码这件事，浏览器自己也已经会做了。把容器解析交回给 JS，把解码交回给浏览器，剩下的体积、启动速度和可控性都会回来。

⊗ 通常的做法

- ffmpeg 编译成 wasm，首屏得先等几十 MB 下载完
- 解码跑在 wasm 单线程里，长视频掉帧、发热明显
- 能力全量打包，只播一个 MP4 也要带上所有容器代码

✔ pure-webcodecs-player

- 零构建纯 ESM，按需 import 单个容器模块，用多少取多少
- VideoDecoder 走浏览器原生解码通路，帧直送 Canvas，不抢主线程
- 解复用自己写，帧级可控：seek、切轨、时间戳全部整数 μs

PIPELINE — 一条管线走到底

16 种容器，共用同一套解析契约

每个模块都是同一个基类契约的实现：open() 拿 MediaInfo，readSample() 取帧，seek() 对齐关键帧。换容器不换调用方式。

01 ⬇️ 输入 Input 任何字节源都能接：本地 File、带 Range 的 HTTP、WS 网关桥接、WebRTC 流。 File · HTTP Range · WS · WebRTC	02 🔗 解复用 Demux 自研容器解析器输出标准 EncodedChunk，时间戳统一为整数 μs，没有浮点漂移。 mp4 / ts / hls / mkv / flv ...	03 ⚙️ 解码 Decode 优先 WebCodecs 硬解；浏览器不支持或格式受限，自动降级到 MSE 路线。 VideoDecoder · AudioDecoder · MSE	04 🖼️ 渲染 Render 视频帧上 Canvas / WebGPU，音频交给 AudioWorklet 做采样级对齐。 Canvas · WebGPU · AudioWorklet
---	---	--	--

CAPABILITIES — 三条硬能力

帧在你手里，而不是在黑盒里

01 — WebCodecs 主路线

自研解复用 + 原生解码，帧级可控

解出来的帧直接交到你手上：Canvas 绘制、滤镜、截图、推理前处理都随你安排，不用再从 <video> 里反抽像素。

- ✔ VideoDecoder / AudioDecoder 原生调用，帧回调可订阅
- ✔ firstframe / error 等事件齐全，seek 关键帧对齐

WebCodecs 解码实拍

MSE 兜底路线

02 — MSE 兜底降级

能硬解就硬解，不能就平滑落地

一条 routePreference 声明优先级，WebCodecs 不可用或格式受限，自动切 MSE 走 <video>，业务代码一行不用改。

- ✔ routePreference: ['webcodecs', 'mse'] 顺序可配
- ✔ 降级对上层透明，事件与 API 完全一致

03 — 零构建零依赖

仓库即产物，jsdelivr 直接当包源

没有 bundler、没有转译、没有 node_modules。浏览器原生 ESM 直接跑，改一行源码刷新即生效，调试栈里全是自己的代码。

- ✔ 纯 ESM 单包，16 个模块按子路径按需引入
- ✔ npm / jsdelivr CDN / GitHub Pages 三种引入任选

零依赖源码结构

MODULES — 格式矩阵

16 个容器与协议模块，一个都不外包

每个模块都能单独引：import * as mp4Mod from 'pure-webcodecs-player/mp4'。同一套基类契约，换容器不换调用方式。

core 播放调度内核：createPlayer、解码路由、事件总线	mp4 ISO BMFF 盒解析，moov 索引 + HTTP Range 拉流	ts MPEG-TS 188 字节包，PAT / PMT 重建轨道	nls m3u8 解析 + 分片调度，直播与点播同构
cmaf CMAF / fMP4 分片，与 DASH、HLS 共用媒体段	flv FLV tag 解析，覆盖 AVC / AAC 与脚本 tag	mkv EBML 解析，Matroska 与 WebM 通吃	mov QuickTime 容器，与 MP4 共享盒结构方言
flac 无损音频解析，seekTable 精准定位	ape Monkey's Audio 无损，帧级 seek 支持	wav RIFF / WAVE，PCM 与扩展格式块	subtitle SRT / VTT 解析与时间轴挂载
webtorrent createSource 走 P2P，边下边播	webrtc SDP 协商与实时媒体流接入	rtmp WebSocket 网关桥接（浏览器无 TCP API）	rtsp WS 网关 / WebRTC 转封装桥接

GET STARTED — 三行起播

createPlayer 一条龙，剩下交给管线

注册你要用的容器模块，剩下的探测、选路、缓冲、解码、渲染由 Player 统一调度。想更底层一点，也可以直接 new 一个 Demux 自己拉帧。

不是效果图，是跑出来的

970 / 970 单元测试全部通过 (Node ≥ 22)

15 步 真机 e2e 自断言：load → 首帧 → seek → destroy

9 第三方运行时依赖，全部浏览器原生 API

所有容器 / 传输模块 API 同构，形状与差异见 docs/CONTRACTS.md

```
player.js

// 注册容器模块 URL 自动探测容器并选路
import { createPlayer, registerDemuxer } from 'pure-webcodecs-player';
import * as mp4Mod from 'pure-webcodecs-player/mp4';

registerDemuxer(mp4Mod);

const player = await createPlayer({
  url: 'https://example.com/movie.mp4',
  canvas: document.getElementById('stage'),
  routePreference: ['webcodecs', 'mse'],
});

await player.load();
await player.play(); // 起播前向缓冲 3s 后出水
player.on('firstframe', () => console.log('首帧已出'));
```

把播放器这件事，交回给浏览器

打开在线演示，拖一个本地 .mp4 / .ts 进去——解复用、解码、渲染全在你自己的浏览器里跑，数据一步都不外出。

打开在线演示

读 README

MIT License · 零构建纯 ESM · Node ≥ 22 · 无第三方运行时依赖